

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.svm import SVC
import matplotlib.pyplot as plt
```

```
# 1-Dataset Preparation
breast_cancer = load_breast_cancer()

X = breast_cancer.data
y = breast_cancer.target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# 2-Linear Models : 2.1 Perceptron
model_perceptron = Perceptron(max_iter=1000, eta0=1)

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model_perceptron.fit(X_train_scaled, y_train)

y_pred_perceptron = model_perceptron.predict(X_test_scaled)
print("Accuracy of perceptron:", accuracy_score(y_test, y_pred_perceptron))
```

Accuracy of perceptron: 0.9649122807017544

```
# 2.2 Logistic Regression
model_logistic_regression = LogisticRegression(max_iter=100)

model_logistic_regression.fit(X_train_scaled, y_train)
y_pred_logistic_regression = model_logistic_regression.predict(X_test_scaled)

print("Accuracy of logistic regression:", accuracy_score(y_test, y_pred))
print("Confusion Matrix of logistic regression: \n", confusion_matrix(y_test, y_pred_logistic_regression))
```

Accuracy of logistic regression: 0.9824561403508771

Confusion Matrix of logistic regression:

```
[[ 62   1]
 [  2 106]]
```

```
# 2.3 Linear SVM
model_svm = SVC(kernel='linear')
model_svm.fit(X_train_scaled, y_train)
y_pred_svm = model_svm.predict(X_test_scaled)
print("Accuracy of SVM:", accuracy_score(y_test, y_pred_svm))

print("Confusion Matrix of SVM: \n", confusion_matrix(y_test, y_pred_svm))
```

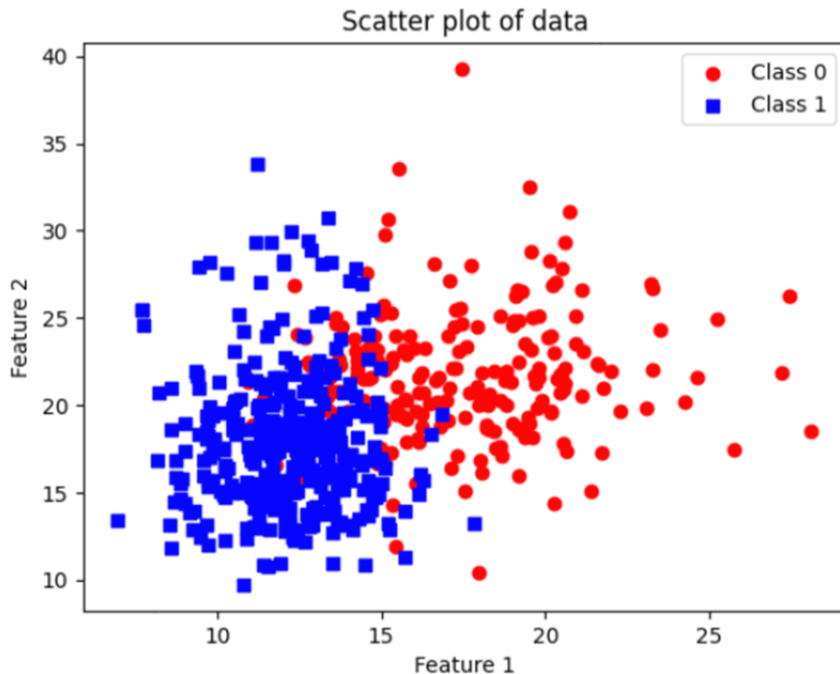
Accuracy of SVM: 0.9766081871345029

Confusion Matrix of SVM:

```
[[ 61  2]
 [ 2 106]]
```

- 4 Compare the results among linear models : Logistic Regression > SVM > Perceptron in terms of accuracy. Logistic Regression is slightly better in minimizing false positives.
- compare with tp4 : Linear models are fast, interpretable, but limited to linear separation. Non-linear models (KNN, Decision Tree, Random Forest) can capture more complex patterns, often giving slightly higher accuracy than simple linear models. Random Forest is often best for tabular data, balancing accuracy and generalization.

```
plt.scatter(X[y==0,0], X[y==0,1], color='red', marker='o', label='Class 0')
plt.scatter(X[y==1,0], X[y==1,1], color='blue', marker='s', label='Class 1')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.title('Scatter plot of data')
plt.show()
```



- 5 Questions :
 - Standardization helps linear models learn properly and faster.
 - It assigns points on one side to Class 1 and the other side to Class 0.
 - Perceptron only works for linearly separable data.
 - Perceptron : class only / Logistic Regression : class + probability.
 - Linear SVM usually gives better generalization and more stable boundaries than Perceptron.
 - Linear models → best for simple, linearly separable data. KNN / Trees / Random Forest → better for complex, non-linear data.